Sesion 7

Regla de la Cadena y Derivadas de Orden Superior

Carlos Cesar Sanchez Coronel

CALCULO Y ALGEBRA LINEAL PARA IA

Contenido

1. Composicion de Funciones
2. Regla de la Cadena
3. Aplicacion en IA: Backpropagation
4. Derivadas de Orden Superior
5. Ejemplos con Python
6. Notacion en Papers Cientificos
7. Ejercicios Propuestos
8. Resumen

Contenido

1. Composicion de Funciones
2. Regla de la Cadena
3. Aplicacion en IA: Backpropagation
4. Derivadas de Orden Superior
5. Ejemplos con Python
6. Notacion en Papers Cientificos
7. Ejercicios Propuestos
8. Resumen

Funcion compuesta

Definicion

Dadas dos funciones f y g, la composicion $f \circ g$ se define como

$$(f \circ g)(x) = f(g(x))$$

Primero se aplica g, luego f al resultado.

Ejemplo

Sean $g(x) = x^2 + 1$ y $f(u) = \sin u$. Entonces

$$h(x) = f(g(x)) = \sin(x^2 + 1)$$

- Las redes neuronales son composiciones enormes de funciones.

Contenido

1. Composicion de Funciones
2. Regla de la Cadena
3. Aplicacion en IA: Backpropagation
4. Derivadas de Orden Superior
5. Ejemplos con Python
6. Notacion en Papers Cientificos
7. Ejercicios Propuestos
8. Resumen

Formulaciones de la regla de la cadena

Forma de Leibniz

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Forma de Lagrange

$$(f \circ g)'(x) = f'(g(x)) \cdot g'(x)$$

Ejemplo

Derivar $h(x) = \sin(x^2 + 1)$.

$$u = x^2 + 1, \quad h = \sin u$$
$$h'(x) = \cos u \cdot (2x) = 2x \cos(x^2 + 1)$$

Regla de la cadena generalizada

Para composiciones multiples, se aplica sucesivamente:

$$\frac{d}{dx} f(g(h(x))) = f'(g(h(x))) \cdot g'(h(x)) \cdot h'(x)$$

Ejemplo con tres niveles

Derivar $k(x) = e^{\cos(x^2)}$.

$$\begin{aligned} a = x^2, \quad b = \cos a, \quad k = e^b \\ k'(x) &= e^b \cdot (-\sin a) \cdot 2x \\ &= -2xe^{\cos(x^2)} \sin(x^2) \end{aligned}$$

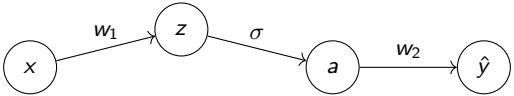
Contenido

1. Composicion de Funciones
2. Regla de la Cadena
3. Aplicacion en IA: Backpropagation
4. Derivadas de Orden Superior
5. Ejemplos con Python
6. Notacion en Papers Cientificos
7. Ejercicios Propuestos
8. Resumen

Red neuronal simple

- Una entrada x, una capa oculta con una neurona (activacion sigmoide) y una salida lineal.

$$
\begin{aligned}
z &= w_1 x + b_1 \\
a &= \sigma(z) = \frac{1}{1 + e^{-z}} \\
\hat{y} &= w_2 a + b_2 \\
L &= \frac{1}{2}(\hat{y} - y)^2 \quad \text{(MSE)}
\end{aligned}
$$



Calculo de gradientes por regla de la cadena

Derivadas parciales

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} = (\hat{y} - y)\, a$$

$$\frac{\partial L}{\partial b_2} = \hat{y} - y$$

Para w_1:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

donde $\frac{\partial \hat{y}}{\partial a} = w_2$, $\frac{\partial a}{\partial z} = a(1-a)$, $\frac{\partial z}{\partial w_1} = x$.

Ejemplo numerico paso a paso

Datos: $x = 2,\ w_1 = 0{,}5,\ b_1 = 0{,}1,\ w_2 = 1{,}0,\ b_2 = 0{,}0,\ y = 5{,}0$

$$z = 0{,}5 \cdot 2 + 0{,}1 = 1{,}1$$

$$a = \sigma(1{,}1) = \frac{1}{1 + e^{-1{,}1}} \approx 0{,}75026$$

$$\hat{y} = 1{,}0 \cdot 0{,}75026 + 0{,}0 = 0{,}75026$$

$$L = \frac{1}{2}(0{,}75026 - 5)^2 \approx 9{,}031$$

$$\frac{\partial L}{\partial \hat{y}} = -4{,}24974$$

$$\frac{\partial L}{\partial w_2} = -4{,}24974 \times 0{,}75026 \approx -3{,}188$$

$$\frac{\partial L}{\partial w_1} = (-4{,}24974) \times 1{,}0 \times 0{,}75026(1 - 0{,}75026) \times 2 \approx -1{,}593$$

Contenido

1. Composicion de Funciones
2. Regla de la Cadena
3. Aplicacion en IA: Backpropagation
4. Derivadas de Orden Superior
5. Ejemplos con Python
6. Notacion en Papers Cientificos
7. Ejercicios Propuestos
8. Resumen

Segunda derivada y curvatura

Definicion

$f''(x) = \frac{d}{dx}\left(\frac{df}{dx}\right) = \frac{d^2 f}{dx^2}$

- $f''(x) > 0$: funcion convexa (curva hacia arriba).
- $f''(x) < 0$: funcion concava (curva hacia abajo).
- $f''(x) = 0$: posible punto de inflexion.

Ejemplo

$f(x) = x^3 - 3x$:

$$f'(x) = 3x^2 - 3, \quad f''(x) = 6x$$

En $x = 1$: $f''(1) = 6 > 0$ (minimo). En $x = -1$: $f''(-1) = -6 < 0$ (maximo). En $x = 0$: $f''(0) = 0$ (punto de inflexion).

Matriz Hessiana

Para funciones de varias variables

La matriz Hessiana **H** contiene las segundas derivadas parciales:

$$\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

- En optimizacion, el Hessiano indica la curvatura de la funcion de perdida.
- Metodos de segundo orden (Newton) usan el Hessiano para acelerar convergencia.
- Los puntos de silla (comunes en altas dimensiones) tienen Hessiano indefinido.

Contenido

1. Composicion de Funciones
2. Regla de la Cadena
3. Aplicacion en IA: Backpropagation
4. Derivadas de Orden Superior
5. Ejemplos con Python
6. Notacion en Papers Cientificos
7. Ejercicios Propuestos
8. Resumen

Derivadas de orden superior con SymPy

```
import sympy as sp

x = sp.Symbol('x')
f = x**3 * sp.exp(x) - sp.ln(x)

# Primera derivada
f1 = sp.diff(f, x)
print("f'(x) =", f1)

# Segunda derivada
f2 = sp.diff(f, x, 2)
print("f''(x) =", f2)

# Evaluar en un punto
valor = f2.subs(x, 1).evalf()
print("f''(1) =", valor)
```

Derivadas parciales y Hessiano simbolico

```
import sympy as sp

x, y = sp.symbols('x y')
f = x**2 * y + sp.sin(y)

# Derivadas parciales
df_dx = sp.diff(f, x)
df_dy = sp.diff(f, y)
print("df/dx =", df_dx)
print("df/dy =", df_dy)

# Hessiano
hess = sp.hessian(f, (x, y))
print("Hessiano:\n", hess)
```

Backpropagation manual en Python

```python
import numpy as np

x, y_true = 2.0, 5.0
w1, b1 = 0.5, 0.1
w2, b2 = 1.0, 0.0

# Forward
z = w1 * x + b1
a = 1 / (1 + np.exp(-z))
y_pred = w2 * a + b2
loss = 0.5 * (y_pred - y_true)**2

print(f"Prediccion: {y_pred:.4f}, Loss: {loss:.4f}")

# Backward
dL_dy = y_pred - y_true
dy_da = w2
da_dz = a * (1 - a)
dz_dw1 = x
dz_db1 = 1.0
grad_w1 = dL_dy * dy_da * da_dz * dz_dw1
grad_b1 = dL_dy * dy_da * da_dz * dz_db1
grad_w2 = dL_dy * a
grad_b2 = dL_dy

print("Gradientes:", grad_w1, grad_b1, grad_w2, grad_b2)
```

Verificacion con JAX (diferenciacion automatica)

```python
import jax
import jax.numpy as jnp

def forward(params, x):
    w1, b1, w2, b2 = params
    z = w1 * x + b1
    a = 1 / (1 + jnp.exp(-z))
    y_pred = w2 * a + b2
    return y_pred

def loss(params, x, y_true):
    y_pred = forward(params, x)
    return 0.5 * (y_pred - y_true)**2

params = (0.5, 0.1, 1.0, 0.0)
x_val, y_val = 2.0, 5.0

grad_loss = jax.grad(loss)
grads = grad_loss(params, x_val, y_val)
print("Gradientes con JAX:", grads)
```

Contenido

1. Composicion de Funciones
2. Regla de la Cadena
3. Aplicacion en IA: Backpropagation
4. Derivadas de Orden Superior
5. Ejemplos con Python
6. Notacion en Papers Cientificos
7. Ejercicios Propuestos
8. Resumen

Notacion comun en Deep Learning

- $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)}(\mathbf{a}^{(l-1)})^\top$: gradiente de la perdida respecto a pesos de la capa l.
- $\delta^{(l)}$: error propagado hacia atras.
- $\mathbf{H} = \nabla^2_{\mathbf{w}}\mathcal{L}(\mathbf{w})$: matriz Hessiana.
- Actualizacion de Newton: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{H}^{-1}\nabla\mathcal{L}(\mathbf{w}_t)$.

Contenido

1. Composicion de Funciones
2. Regla de la Cadena
3. Aplicacion en IA: Backpropagation
4. Derivadas de Orden Superior
5. Ejemplos con Python
6. Notacion en Papers Cientificos
7. Ejercicios Propuestos
8. Resumen

Ejercicios

1. Deriva $f(x) = \ln(\sin(e^x))$ usando regla de la cadena paso a paso.
2. Calcula la segunda derivada de $g(x) = e^{x^2+1}$.
3. Para la red neuronal del ejemplo, obtén las derivadas parciales si la activacion fuera ReLU en lugar de sigmoide. ¿Que cambios observas?
4. Implementa en Python una red con dos neuronas en la capa oculta y calcula los gradientes manualmente (usa un bucle sobre las neuronas).
5. Investiga: ¿que es el "vanishing gradientz como se relaciona con la regla de la cadena?

Contenido

1. Composicion de Funciones
2. Regla de la Cadena
3. Aplicacion en IA: Backpropagation
4. Derivadas de Orden Superior
5. Ejemplos con Python
6. Notacion en Papers Cientificos
7. Ejercicios Propuestos
8. Resumen

Lo que hemos aprendido

Conceptos clave

- **Regla de la cadena**: derivada de funciones compuestas, base del backpropagation.
- **Aplicacion en IA**: calculo de gradientes en redes neuronales simples.
- **Derivadas de orden superior**: curvatura, segunda derivada, matriz Hessiana.
- **Optimizacion**: criterios de minimos/maximos y metodos de segundo orden.
- **Implementacion**: SymPy para derivadas simbolicas, NumPy para backprop manual, JAX para diferenciacion automatica.

Conexion con futuros cursos

- **Deep Learning**: backpropagation en redes profundas.
- **Optimizacion avanzada**: metodos de Newton, quasi-Newton.
- **Teoria del aprendizaje**: analisis de convergencia y puntos criticos.

Gracias!